Math-as-Structural-Compression for AI Solving

A Full Engineering Specification using SEF/UTC (frame discipline) + DCIP (modular optimality)

Kevin T.N
jkdkr2439@gmail.com

February 19, 2026

Abstract

This document is a max-option technical specification for building an AI math solver under the thesis: *mathematics is a structural compression language*. The solver therefore must not merely map tokens to tokens; it must (1) decompress problem statements into typed relational structure, (2) operate on that structure using domain-specific modules and verified transformations, and (3) re-compress the result into a human-readable or proof-checkable solution.

We integrate: **SEF/UTC** to prevent frame conflation (explicit Reference Frame, typed definitions, and admissible inference), and **DCIP** to scale via block-sparse interactions (modular coupling with explicit interfaces and an interior optimum granularity).

Deliverables include: architecture, module registry standard, data contracts, DCIP router + proof planning/search algorithm, verification pipelines, telemetry metrics (SR/EE/coupling), evaluation plan, failure modes, and multi-domain worked examples.

Contents

1 Executive Overview (what you can actually build) 3
1.1 Core claim . . . 3
1.2 Why DCIP is the scaling law here . . . 3
1.3 Build target . . . 3

2 SEF/UTC Discipline (frame, types, and definition as existence) 3
2.1 Reference Frame (RF) and strata discipline . . . 3
2.2 SEF triple: RF–ID–DEF . . . 4
2.3 Why this matters (preventing frame conflation) . . . 4

3 DCIP Formal Model (costs, coupling, and optimal modularity) 4
3.1 System as interacting processes . . . 4
3.2 Costs under bounded capacity . . . 4
3.3 DCIP design takeaway . . . 5

4 Architecture: components, APIs, and contracts 5
4.1 Component diagram (engineering view) . . . 5
4.2 Core data contract: "Problem IR" . . . 5

5 Math domain modules (registry standard) 5
5.1 Module interface contract (I1–I6) . . . 5
5.2 Module manifest template (what engineers implement) . . . 6

6 Clean diagrams (no spaghetti) **6**
6.1 Rule: collapse universal dependencies into buses . . . 6
6.2 Figure A: modules only (max readability) . . . 7
6.3 Figure B: bridges only (routing graph) . . . 7

7 Problem IR (full schema) and data contracts **7**
7.1 IR overview . . . 7
7.2 Full IR schema (engineering-ready) . . . 7

8 DCIP Router (domain selection + tactic shortlist) **9**
8.1 Signal-driven scoring . . . 9
8.2 Interface legality . . . 9

9 Planner/Search (how solutions are actually found) **10**
9.1 Hierarchical planning (block-sparse proof search) . . . 10
9.2 Plan scoring . . . 10

10 Engines and Verification (what makes it reliable) **11**
10.1 Engine stack . . . 11
10.2 Certificates . . . 11
10.3 Verification gates (SEF enforcement) . . . 11

11 Telemetry: SR/EE/Coupling metrics (debugging the solver) **11**
11.1 Coupling metrics (DCIP) . . . 11
11.2 SR proxies (Structural Resonance) . . . 11
11.3 EE proxies (Epistemic Entropy) . . . 12

12 Evaluation plan (what experiments prove DCIP helps) **12**
12.1 Bench categories . . . 12
12.2 Ablations . . . 12
12.3 Key success metrics . . . 12

13 Failure modes (DCIP stress map) and mitigations **12**
13.1 Failure mode: monolithic overload . . . 12
13.2 Failure mode: over-modularization . . . 12
13.3 Failure mode: sparse-but-wrong routing . . . 12
13.4 Failure mode: deep dependency amplification . . . 13

14 Worked examples (multi-domain) **13**
14.1 Example 1: OPT → A1 (inequality route) . . . 13
14.2 Example 2: A2/G1 (compactness route) . . . 13
14.3 Example 3: CMB → OPT (flow/matching) . . . 13

15 Implementation notes (practical engineering choices) **13**
15.1 Minimal viable build (MVB) . . . 13
15.2 Where LLMs fit (and where they must not) . . . 13

A Appendix A: Module selection signals (starter list) **14**

B Appendix B: Engineering checklist (SEF + DCIP) **14**

1 Executive Overview (what you can actually build)

1.1 Core claim

The solver is a **structure-first** system:

$$\text{Problem Text} \to \text{Typed Structure} \to \text{Modular Reasoning} \to \text{Verified Output}$$

Not:

$$\text{Problem Text} \to \text{Token Completion.}$$

1.2 Why DCIP is the scaling law here

As problem depth grows (more lemmas, more dependencies, more domains), a monolithic attention system faces:

- uncontrolled cross-talk (dense coupling),
- error cascades (mis-typed objects poison later steps),
- coordination overhead (search branching explodes),
- brittleness (no fault containment).

DCIP predicts that performance improves when reasoning is **block-sparse**: strong intra-domain coherence, minimal inter-domain coupling, and only via explicit interfaces. Over-fragmentation has its own cost; hence an interior optimal modularity often exists.

1.3 Build target

A practical build can be done as a modular pipeline:

1. **Parser & Normalizer** (text → AST-like form)
2. **Type Inference & Structure Extraction** (AST → typed constraint graph)
3. **DCIP Router** (choose domain modules + tactics shortlist with coupling budget)
4. **Planner/Search** (compose lemmas/tactics, controlled branching)
5. **Compute Engine** (CAS + numeric + SMT where appropriate)
6. **Verifier** (proof checker / certificate verifier / consistency checks)
7. **Renderer** (human explanation + optionally proof object)
8. **Telemetry** (SR/EE/coupling metrics + failure diagnostics)

2 SEF/UTC Discipline (frame, types, and definition as existence)

2.1 Reference Frame (RF) and strata discipline

Definition 1 (Reference Frame of this system)**.** We work in an operational, intersubjective modeling frame: math as a system of structures, invariants, and admissible transformations used by bounded agents. This is not an ontological claim about reality.

Remark 1 (UTC strata discipline)**.** We keep reasoning in the structural/intersubjective layer (R2). Any metaphysical jump (R3) is explicitly labeled as conjectural and never required for system correctness.

2.2 SEF triple: RF–ID–DEF

Definition 2 (SEF triple)**.** Every object/concept used by the solver must be represented as:

$$(\mathrm{RF}, \mathrm{ID}, \mathrm{DEF}),$$

where DEF includes:

- **Type signature** (sorts/structures),
- **Relational invariants** (what must remain true),
- **Admissible transformations** (how it can change without breaking meaning).

2.3 Why this matters (preventing frame conflation)

Most solver failures are **type/frame errors** disguised as algebra mistakes:

- applying a lemma with hidden preconditions (e.g., convexity, completeness),
- mixing discrete/continuous assumptions,
- using a theorem requiring Choice/Excluded Middle in a constructive RF.

SEF forces explicit declaration of the frame and admissible rules.

3 DCIP Formal Model (costs, coupling, and optimal modularity)

3.1 System as interacting processes

Definition 3 (Coupling matrix)**.** Let the solver be composed of processes $P_1, \ldots, P_n$ (parser, typer, router, module engines, verifier, etc.). Define a weighted coupling matrix $\boldsymbol{C} = [c_{ij}]$, where $c_{ij} \geq 0$ measures how much the output of process i affects the input of process j.

3.2 Costs under bounded capacity

Definition 4 (Coordination cost)**.**

$$E_{\mathrm{coord}}(\boldsymbol{C}) \;\propto\; \sum_{i \neq j} c_{ij}.$$

Interpretation: cross-component coordination, glue code, context passing, and branching management.

Definition 5 (Error propagation cost (spectral proxy))**.**

$$E_{\mathrm{err}}(\boldsymbol{C}) \;\propto\; \phi(\rho(\boldsymbol{C})),$$

where $\rho(\boldsymbol{C})$ is the spectral radius and ϕ is increasing (optionally explosive near stability thresholds). Interpretation: feedback loops and long dependency chains amplify small errors.

Definition 6 (Abstraction and interface costs)**.** Modularization induces:

$$E_{\mathrm{abs}} \sim \mathrm{H}(X \mid Z), \qquad E_{\mathrm{iface}} \sim g(k),$$

where Z is the coarse-grained interface state and k is the number of modules used.

Assumption 1 (Bounded capacity)**.** The solver has finite budgets (time, compute, search width, token budget). Total effective cost must stay below $K < \infty$.

3.3 DCIP design takeaway

Dense coupling increases both coordination cost and error cascades. Modularization reduces coupling but increases interface/abstraction overhead; therefore an interior optimal modularity often exists.

4 Architecture: components, APIs, and contracts

4.1 Component diagram (engineering view)

We build the solver as a set of services/modules with explicit contracts:

- **C0: Input Gateway** (text/latex/markdown to normalized format)
- **C1: Parser** (normalized text → syntax tree)
- **C2: Type Inference** (syntax tree → typed objects)
- **C3: Structure Extractor** (typed objects → constraint graph + goal)
- **C4: Module Registry** (list of domains, interfaces, lemma signatures, tactics)
- **C5: DCIP Router** (select modules + bridging paths + tactics shortlist)
- **C6: Planner/Search** (compose steps; controls branching; generates a plan)
- **C7: Engines** (CAS / numeric / SMT / theorem prover / graph algorithms)
- **C8: Verifier** (proof checker or certificate verifier; type/condition checks)
- **C9: Renderer** (human explanation + optional proof object)
- **C10: Telemetry** (SR/EE/coupling metrics, traces, failure diagnosis)

4.2 Core data contract: "Problem IR"

All components communicate through a single intermediate representation (IR). This is the backbone of the engineering spec.

IR must contain

- Typed objects
- Constraints (as a graph)
- Goal specification
- Candidate domain modules (with scores)
- Interface-legal reduction paths between modules
- Budget controls (DCIP coupling budget, branching limits)

5 Math domain modules (registry standard)

5.1 Module interface contract (I1–I6)

Definition 7 (Module contract (required fields))**.** Each domain module must declare:

1. **I1 Types/Sorts**: what objects it owns (e.g., Real, Graph, MeasureSpace)

2. **I2 Morphisms/Transforms**: admissible transformations (substitution, homomorphism, continuous map)
3. **I3 Invariants**: key invariants (norm, rank, convexity, connectivity, measure)
4. **I4 Reductions**: interface-legal bridges to other modules (conditions + certificates)
5. **I5 Dualities/Correspondences**: optional equivalences (e.g., LP duality, Fourier)
6. **I6 Computation hooks**: algorithms, tactic templates, and certificates/verifiers

5.2 Module manifest template (what engineers implement)

A module is shipped as a manifest + implementation hooks.

```
{
  "module_id": "A2",
  "name": "Analysis (Continuous)",
  "types": ["Real", "Function", "MetricSpace", "NormedSpace"],
  "invariants": ["continuity", "boundedness", "convergence", "convexity"],
  "morphisms": ["differentiate", "integrate", "limit", "substitute"],
  "lemmas": [
    {"id":"EVT", "name":"Extreme Value Theorem",
     "requires":["continuity","compact_domain"], "provides":["attains_max_min"]},
    {"id":"JENSEN", "name":"Jensen Inequality",
     "requires":["convexity"], "provides":["bound"]}
  ],
  "reductions": [
    {"to":"OPT", "via":"KKT/Lagrange", "requires":["differentiable"]},
    {"to":"P1", "via":"measure/integration", "requires":["measurable"]}
  ],
  "computation_hooks": [
    {"hook":"derivative_engine", "certificate":"symbolic_check"},
    {"hook":"numeric_checker", "certificate":"interval_bounds"}
  ]
}
```

6 Clean diagrams (no spaghetti)

6.1 Rule: collapse universal dependencies into buses

In diagrams, do not draw L0 → every node and CAT → every node. Those are universal. Instead:

- draw one arrow from L0 into the **domain cluster** (a bus),
- draw one dashed arrow from CAT into the domain cluster (meta bus),
- draw only the **high-value bridges** explicitly.

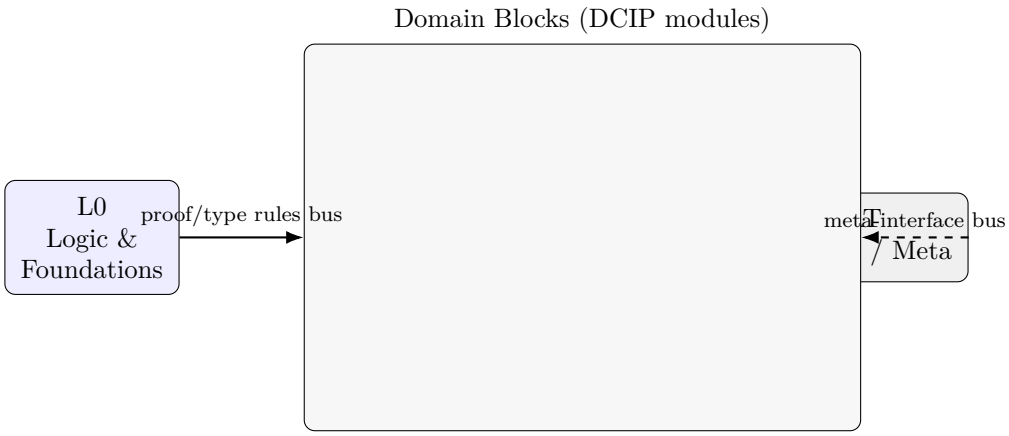

Figure 1: Figure A (modules only): universal dependencies shown as buses into the domain cluster.

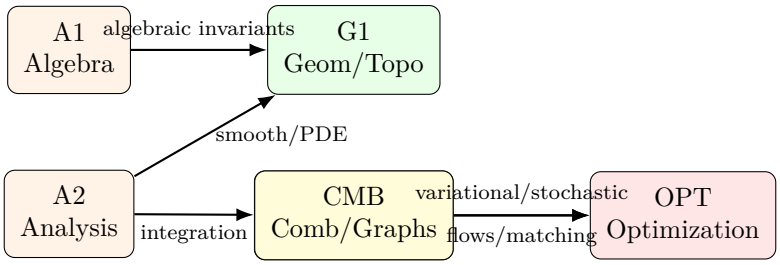

Figure 2: Figure B (bridges only): the explicit inter-domain routing graph. Engineers can treat this as a legal move graph.

6.2 Figure A: modules only (max readability)

6.3 Figure B: bridges only (routing graph)

7 Problem IR (full schema) and data contracts

7.1 IR overview

The IR is a typed constraint hypergraph + goal + routing controls. This enables:

- explicit type checking,
- interface-legal module transitions,
- controlled proof search,
- verifiable certificates.

7.2 Full IR schema (engineering-ready)

```
{
  "rf": {
    "foundation": "ZFC | TypeTheory | HoTT",
    "logic": "Classical | Intuitionistic",
    "assumption_stack": ["Choice?", "ExcludedMiddle?", "Completeness?"
        ],
    "resource_bounds": {
      "time_budget_ms": 10000,
```

```
      "max_steps": 5000,
      "max_branching": 30,
      "max_cross_module_hops": 3
    }
  },

  "problem": {
    "source_text": "string (optional)",
    "objects": [
      {
        "id": "x",
        "type": "Real | Integer | Graph | Function | Set | MeasureSpace | ...",
        "structure_tags": ["OrderedField", "MetricSpace", "InnerProductSpace", "..."],
        "properties": ["x>0", "x in [0,1]", "..."]
      }
    ],
    "constraints": [
      {
        "id": "c1",
        "kind": "Equation | Inequality | Membership | Definition | Predicate",
        "expr": "x^2 + y^2 = 1",
        "scope": ["x", "y"],
        "certificate": null
      }
    ],
    "goal": {
      "kind": "Prove | Compute | Optimize | Find | Exists | ForAll",
      "expr": "maximize x+y",
      "output_type": "Real | ProofObject | Certificate"
    }
  },

  "structure_extraction": {
    "signals": [
      {"signal":"optimize", "weight":0.95},
      {"signal":"quadratic_constraint", "weight":0.80},
      {"signal":"symmetry", "weight":0.60}
    ],
    "typed_constraint_graph": {
      "nodes": [
        {"node":"x","type":"Real"},
        {"node":"y","type":"Real"},
        {"node":"c1","type":"Constraint"},
        {"node":"goal","type":"Objective"}
      ],
      "edges": [
        {"from":"c1","to":"x","role":"involves"},
        {"from":"c1","to":"y","role":"involves"},
        {"from":"goal","to":"x","role":"depends"},
        {"from":"goal","to":"y","role":"depends"}
      ]
    }
  },

  "routing": {
```

```
    "candidate_modules": [
      {"module":"OPT","score":0.92},
      {"module":"A1","score":0.70},
      {"module":"A2","score":0.62}
    ],
    "allowed_modules": ["OPT","A1","A2"],
    "bridge_graph": [
      {"from":"OPT","to":"A1","via":"Cauchy-Schwarz", "requires":["InnerProductSpace"]},
      {"from":"OPT","to":"A2","via":"Lagrange", "requires":["Differentiable"]}
    ],
    "dcip_budget": {
      "max_cross_module_hops": 3,
      "hop_penalty": 0.2,
      "max_case_splits": 6
    }
  },

  "plan": {
    "lemmas": [],
    "tactics": [],
    "search_trace": []
  },

  "verification": {
    "mode": "light | heavy",
    "proof_checker": "none | lean | coq | isabelle",
    "certificate_checks": ["typecheck", "preconditions", "numeric_bounds"]
  },

  "telemetry": {
    "sr_proxy": null,
    "ee_proxy": null,
    "coupling_metrics": null
  },

  "rendering": {
    "format": "human_readable | formal_proof | both",
    "explain_interfaces": true
  }
}
```


8 DCIP Router (domain selection + tactic shortlist)

8.1 Signal-driven scoring

We compute scores for modules using weighted structural signals:

$$s(m) = \sum_k w_k \cdot \mathbf{1}\{\text{signal}_k \in m\} - \lambda \cdot \text{interfaceCost}(m)$$

Then select top-M modules under a coupling budget.

8.2 Interface legality

A move $m \to m'$ is legal only if:

- there is a declared reduction/duality edge in the bridge graph, and
- its preconditions type-check against current IR.

Algorithm 1: DCIP Router (engineering version)

Input: IR (typed objects, constraints, goal), module registry, bridge graph
Output: Allowed module set, bridge-legal paths, tactics shortlist
Extract structural signals from IR
Score modules using signals + prior success stats + interface overhead penalty
Select top-M modules, respecting budget (max hops, max modules)
Build bridge-legal adjacency (filter by preconditions)
Generate tactic shortlist: invariant-preserving tactics first
Return routing package to Planner/Search

9 Planner/Search (how solutions are actually found)

9.1 Hierarchical planning (block-sparse proof search)

We search in a space of **plans**:

$$\Pi = (t_1, t_2, \ldots, t_T)$$

where each step is a tactic or reduction.

DCIP control is implemented as:

- penalize cross-module hops,
- limit case splits,
- prefer steps that preserve key invariants.

9.2 Plan scoring

A practical scoring function:

$$\text{Score}(\Pi) = \underbrace{\text{Progress}(\Pi)}_{\text{closer to goal}} - \alpha \cdot \#\text{caseSplits} - \beta \cdot \#\text{crossModuleHops} - \gamma \cdot \text{preconditionRisk}$$

where preconditionRisk increases when lemmas are used with weakly-justified assumptions.

Algorithm 2: DCIP-Constrained Proof Search (beam / best-first)

Input: IR, routing package (modules, bridges, tactics), budgets
Output: Plan Π and evidence (certificates/proof objects)
Initialize frontier with empty plan
while *frontier not empty and budget remains* **do**
  Pop best candidate plan
  Expand by applying 1 tactic (or 1 reduction edge)
  Type-check step; attach certificate if available
  Update IR state; compute plan score with DCIP penalties
  Prune if hop budget or branching budget exceeded
  If goal satisfied: return plan + evidence
Return failure + diagnostics (where SR dropped / where EE spiked)

10 Engines and Verification (what makes it reliable)

10.1 Engine stack

Recommended engine composition:

- **Symbolic engine** (CAS): simplification, factoring, derivatives, integrals
- **Discrete engine**: graph algorithms, DP, combinatorial routines
- **SMT/constraint engine**: satisfiability, counterexample search
- **Numeric checker**: interval arithmetic, sanity checks
- **Proof checker (optional heavy mode)**: Lean/Coq/Isabelle

10.2 Certificates

Each nontrivial step should ideally provide a certificate:

- symbolic: transformation trace + rewrite rules
- numeric: interval bounds proof
- graph: flow/cut witness, matching witness
- proof: proof term (heavy mode)

10.3 Verification gates (SEF enforcement)

Before accepting a step:

- type-check objects and lemma preconditions,
- validate that the step is an admissible morphism for the current module,
- validate that cross-module transitions use a declared interface edge.

11 Telemetry: SR/EE/Coupling metrics (debugging the solver)

11.1 Coupling metrics (DCIP)

- cross-module hops count
- bridge density used by plan
- dependency depth (lemma chain depth)

11.2 SR proxies (Structural Resonance)

Operational proxies:

- type alignment rate (how often lemma types match goal types)
- invariant preservation rate (did we keep the key invariants I3?)
- reduction validity rate (interface edges used with satisfied preconditions)

11.3 EE proxies (Epistemic Entropy)

Operational proxies:

- missing-context failures (needed assumption not represented)
- over-coarse summaries (interface state too compressed)
- uncertainty mass in goal-state classifier (router entropy)

12 Evaluation plan (what experiments prove DCIP helps)

12.1 Bench categories

- single-domain short proofs (baseline)
- single-domain deep proofs (lemma chains)
- cross-domain problems (require bridges)
- adversarial type traps (precondition-sensitive)

12.2 Ablations

- remove SEF gating (allow implicit assumptions) $\rightarrow$ expect silent failures
- remove DCIP hop budget $\rightarrow$ expect search blow-up and hallucinated bridges
- remove module routing (monolithic mode) $\rightarrow$ expect degraded scaling

12.3 Key success metrics

- correctness (verified or certificate-valid)
- time/compute cost
- robustness (variance across similar problems)
- interpretability (trace quality, interface explanation)

13 Failure modes (DCIP stress map) and mitigations

13.1 Failure mode: monolithic overload

Symptom: search branching explodes; inconsistent intermediate states. Mitigation: enforce module boundaries; hop penalties; bridge legality checks.

13.2 Failure mode: over-modularization

Symptom: too many tiny modules; interface overhead dominates; solver loses global coherence. Mitigation: learn/estimate interior optimal modularity (tune max modules / hop budget per problem class).

13.3 Failure mode: sparse-but-wrong routing

Symptom: router selects a wrong module; plan becomes consistent but irrelevant. Mitigation: router confidence threshold; fallback to second-best module set; counterexample search.

13.4 Failure mode: deep dependency amplification

Symptom: one mistaken lemma precondition cascades. Mitigation: early verification gates; certificates; reduce dependency depth via stronger lemmas.

14 Worked examples (multi-domain)

14.1 Example 1: OPT → A1 (inequality route)

Problem: maximize $x + y$ subject to $x^2 + y^2 = 1$. Router: OPT high score; bridge OPT→A1 via Cauchy-Schwarz. Plan uses invariant $\|(x, y)\|_2 = 1$:

$$x + y = \langle (1, 1), (x, y) \rangle \leq \|(1, 1)\| \, \|(x, y)\| = \sqrt{2}.$$

Achieved at $x = y = 1/\sqrt{2}$.

14.2 Example 2: A2/G1 (compactness route)

Problem: continuous f on $[a, b]$ attains max/min. Bridge A2→G1 uses compactness of closed intervals in $\mathbb{R}$. Verification gate checks precondition: continuity + compact domain.

14.3 Example 3: CMB → OPT (flow/matching)

Problem: maximum bipartite matching. Bridge CMB→OPT via max-flow/min-cut. Certificate: matching witness or cut witness.

15 Implementation notes (practical engineering choices)

15.1 Minimal viable build (MVB)

If you want a build you can ship fast:

- start with 3 modules: A1, A2, OPT
- implement typing + bridge legality
- implement CAS + numeric checker
- add proof checker later as heavy mode

15.2 Where LLMs fit (and where they must not)

LLMs are best used as:

- parser assistant (suggest structure extraction),
- tactic proposer (candidate step generation),
- explainer (render human narrative).

LLMs must not be the final arbiter of correctness. Verification gates must be non-LLM.

A Appendix A: Module selection signals (starter list)

- OPT: maximize/minimize, constrained, KKT, Lagrange
- A2: limit, continuity, derivative, integral, series
- A1: group/ring, factorization, polynomial identity, linear algebra
- G1: open/closed, compact, connected, manifold
- P1: probability, expectation, variance, entropy
- CMB: graph, matching, flow, counting

B Appendix B: Engineering checklist (SEF + DCIP)

1. All objects typed? (SEF I1)
2. All steps admissible morphisms? (SEF I2)
3. Invariants tracked? (SEF I3)
4. Cross-domain moves legal via bridge graph? (SEF I4/I5)
5. Certificates attached and checked? (Verification)
6. Hop/case budgets enforced? (DCIP)